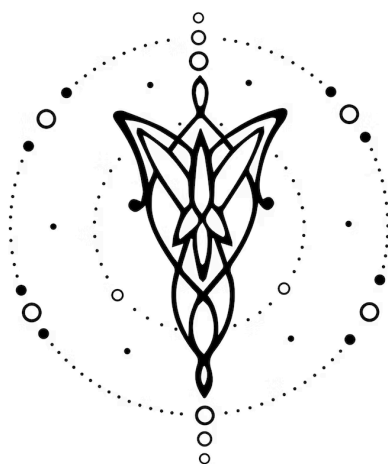




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

8 de Marzo de 2026

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Francisco	Trivelloni	64141	ftrivelloni@itba.edu.ar
Lorenzo	Pizzuto Beltran	64123	lpizzutobeltran@itba.edu.ar

2. Repositorio

La solución será versionada en: <https://github.com/Ftrivelloni/MipASM>.

3. Dominio

Se desarrollará un Lenguaje de Dominio Específico (DSL) procedural, con una sintaxis similar a C/C++, diseñado para la composición algorítmica de música y orientado a programadores sin conocimientos formales en teoría musical. El compilador abstraerá las complejidades matemáticas del tiempo y los eventos de hardware, traduciendo las instrucciones del código a un archivo binario de formato estándar .midi.

Para lograr esto, el lenguaje encapsulará los conceptos musicales (figuras de tiempo, tonos, acordes y familias de instrumentos) en librerías estándar (headers) que el usuario podrá importar según su necesidad, permitiendo generar composiciones complejas, manejar la sincronización de múltiples pistas y alterar las propiedades físicas del sonido (volumen, ataque, paneo) de forma programática.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Un punto de entrada principal obligatorio definido por la función `main()`.
- (II). Se proveerá un sistema de importación de librerías mediante la directiva `#include` para cargar constantes predefinidas de instrumentos, notas, duraciones y efectos.
- (III). Se podrán instanciar múltiples pistas (track), cada una asociada a un instrumento del estándar General MIDI.
- (IV). Se proveerán funciones nativas para emitir sonidos (`play`) y pausas temporales (`rest`) sobre una pista específica.
- (V). Se proveerá un bloque de control semántico `sync {}` que alineará temporalmente la ejecución simultánea de múltiples pistas.
- (VI). Se proveerá una variable de estado global `tempo` que dictaminará la velocidad absoluta de ejecución (Beats Per Minute) para el cálculo de los milisegundos de las figuras musicales.
- (VII). Se proveerán funciones nativas para la manipulación de mensajes Control Change (CC) del protocolo MIDI, tales como `set_volume`, `set_pan` y `set_attack`.
- (VIII). Se proveerán tipos de datos estáticos fundamentales como `int`, `float`, `boolean`, y tipos propios del dominio como `track` y `duration`.
- (IX). Se proveerán operadores relacionales, aritméticos y lógicos estándar (`<`, `>`, `==`, `+`, `-`, `AND`, `OR`, etc.).
- (X). Se proveerán estructuras de control básicas de tipo IF-THEN-ELSE, FOR y WHILE para la repetición y variación algorítmica de los patrones musicales.
- (XI). El lenguaje soportará la escritura de código en su forma "cruda" o primitiva, permitiendo invocar funciones nativas (como `init_track` y `play`) utilizando directamente tipos de datos numéricos fundamentales (`int` para instrumentos/notas y `float` para duraciones relativas) sin necesidad obligatoria de importar librerías estándar.

- (XII). Se proveerá un mecanismo de preprocesamiento para los headers. Las librerías estándar no serán funciones externas, sino archivos escritos en el propio DSL que declaran constantes sobre los tipos de datos primitivos (ej. `const int C4 = 60;`). El compilador inyectará este código antes del análisis sintáctico.

5. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa con una función `main` vacía.
- (II). Un programa que importe un header estándar (ej. `#include < durations.h>`).
- (III). Un programa que instancie una pista (track) asignándole un instrumento válido.
- (IV). Un programa que asigne un valor entero a la variable global de tempo.
- (V). Un programa que emita una nota secuencial usando `play()` en una pista.
- (VI). Un programa que avance el tiempo de una pista en silencio usando `rest()`.
- (VII). Un programa que sincronice dos pistas distintas dentro de un bloque `sync {}`.
- (VIII). Un programa que repita la reproducción de una nota 4 veces utilizando un ciclo `FOR`.
- (IX). Un programa que altere el camino de ejecución musical usando una estructura `IF-ELSE`.
- (X). Un programa que modifique un parámetro de efecto (ej. `set_volume()`) antes de tocar una nota.
- (XI). Un programa que ejecute instrucciones de sonido pasando literales enteros y flotantes crudos en lugar de constantes (ej. `play(t, 60, 0.25)`).
- (XII). Un programa que defina sus propias variables constantes de dominio explícitamente en el archivo principal, sin importar ningún header externo.
- (XIII). Un programa que intente utilizar una variable o constante de dominio (ej. `C4` o `QUARTER_NOTE`) sin haber importado el header correspondiente ni haberla declarado previamente en el código.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa malformado léxicamente.
- (II). Un programa que carezca de la función de entrada `main()`.
- (III). Un programa que intente asignar un tipo de dato incompatible a un track (ej. `track t = "hola";`).
- (IV). Un programa que intente llamar a `play()` pasando un tipo de duración inválido.
- (V). Un programa que posea un bloque `sync { }` sin emparejar correctamente las llaves de apertura y cierre.

6. Ejemplos

En el Cód. (6.1), se puede ver la creación de una secuencia musical básica en una sola pista, utilizando constantes importadas de las librerías estándar para abstraer la notación musical.

```
#include <instruments/pianos.h>
#include <notes.h>
#include <durations.h>

void main() {
    tempo = 120;
    track melody = init_track(ACOUSTIC_GRAND);

    // Reproducción de una pequeña escala con silencios
    play(melody, C4, QUARTER_NOTE);
    play(melody, D4, QUARTER_NOTE);
    rest(melody, HALF_NOTE);
    play(melody, E4, WHOLE_NOTE);
}
```

Código 6.1: Implementación de una melodía secuencial básica con silencios.

En el Cód. (6.2), se procede a crear una composición con dos instrumentos sincronizados. Se demuestra además el uso de estructuras de control (FOR) para crear un bucle rítmico, y la alteración de un efecto de sonido dinámico (set_pan).

```
#include <instruments/guitars.h>
#include <instruments/drums.h>
#include <fx/dynamics.h>
#include <durations.h>

void main() {
    tempo = 140;
    track rhythm_guitar = init_track(DISTORTION_GUITAR);
    track drums = init_track(STANDARD_DRUM_KIT);

    // Paneo de la guitarra al parlante izquierdo
    set_pan(rhythm_guitar, LEFT);

    sync {
        // La batería y la guitarra arrancan al mismo tiempo
        for(int i = 0; i < 4; i = i + 1) {
            play(drums, ROCK_KICK, QUARTER_NOTE);
            play(rhythm_guitar, POWER_CHORD_E, QUARTER_NOTE);
        }
    }
}
```

Código 6.2: Sincronización de múltiples pistas y modificación de efectos de hardware.

En el Cód. (6.3), se demuestra la capacidad del lenguaje para funcionar sin la importación de librerías estándar (headers). El usuario interactúa directamente con los tipos de datos primitivos que espera el backend, introduciendo manualmente los números de instrumento del estándar

MIDI (ej. 30), los valores de tono (ej. 60) y las proporciones matemáticas del tiempo en punto flotante (ej. 0.25).

```
// Programa crudo, sin directivas #include

void main() {
    tempo = 120;

    // Instanciación de pista usando el número de instrumento MIDI explícito
    // (Ej: 30 corresponde a Distortion Guitar)
    track mi_pista = init_track(30);

    // Ejecución usando números enteros para la nota (60 = Do central)
    // y punto flotante para la duración relativa (0.25 = negra)
    play(mi_pista, 60, 0.25);
    play(mi_pista, 64, 0.125);
}
```

Código 6.3: Implementación de secuencias musicales crudas utilizando tipos de datos numéricos primitivos sin depender de la librería estándar.